

ĐẠI HỌC QUỐC GIA, THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



Cấu trúc rời rạc cho khoa học máy tính

Báo cáo bài tập lớn

GVHD: Mai Xuân Toàn
SV thực hiện: Trần Đỗ Cao Trí 2313624

TP Hồ Chí Minh, Tháng 12 Năm 2025



Mục lục

1	Task 1: Shortest Path on Adjacency Matrix	2
1.1	Mục tiêu	2
1.2	Cấu trúc dữ liệu	2
1.3	Heuristic	2
1.4	Thuật toán A*	2
1.5	Hàm <code>findShortestPathMatrix</code>	3
1.6	Tóm tắt	4
2	Task 2: Shortest Path in 2D Space with Two Heuristics	5
2.1	Mục tiêu	5
2.2	Thuật toán	5
2.3	Cài đặt	5
2.4	Tóm tắt	7
3	Task 3: Shortest Path in Maze Using A* With 8-Directional Movement	8
3.1	Mục tiêu	8
3.2	Cấu trúc dữ liệu	8
3.2.1	Node kết quả	8
3.2.2	Node dùng trong A*	8
3.3	Heuristic và kiểm tra hợp lệ	8
3.4	Hướng di chuyển	9
3.5	Thuật toán	9
3.6	Hàm <code>findPathInMaze</code>	9
3.7	Tóm tắt	11
4	Task 4: Biểu Diễn Mê Cung Dưới Dạng Đồ Thị và Tìm Đường A* Trên Ma Trận Trọng Số	12
4.1	Mục tiêu	12
4.2	Cấu trúc dữ liệu	12
4.2.1	Linked-list kết quả	12
4.2.2	Node dùng trong A* cho đồ thị	12
4.3	Chuyển đổi hệ tọa độ	12
4.4	Heuristic	13
4.5	Xây dựng ma trận trọng số	13
4.6	Thuật toán tìm đường A* (trên đồ thị)	13
4.7	Tóm tắt	15



1 Task 1: Shortest Path on Adjacency Matrix

1.1 Mục tiêu

Task 1 yêu cầu cài đặt thuật toán A^* để tìm đường đi ngắn nhất trên đồ thị được biểu diễn bằng ma trận kề 100×100 . Hàm trả về một danh sách liên kết `PathNode` chứa các nút thuộc đường đi cùng giá trị g , h , và f .

1.2 Cấu trúc dữ liệu

```
struct PathNode{
    std::string name;
    double f, g, h;
    PathNode* next;
    PathNode(std::string name,double f,double g,double h)
        : name(name), f(f), g(g), h(h), next(nullptr) {}
};
```

1.3 Heuristic

Heuristic được tính bằng BFS-level: số bước tối thiểu từ một đỉnh đến đỉnh đích trong đồ thị, đảm bảo admissible.

```
int computeHeuristic(int start,int goal,double adj[100][100]){
    vector<int> dist(100,-1); dist[start]=0;
    vector<int> q={start};
    for(int i=0;i<q.size();i++){
        int u=q[i];
        if(u==goal) return dist[u];
        for(int v=0; v<100; v++){
            if(adj[u][v]>0 && dist[v]==-1){
                dist[v]=dist[u]+1; q.push_back(v);
            }
        }
    }
    return 1000000;
}
```

1.4 Thuật toán A^*

A^* duy trì mảng g , h , f và chọn đỉnh chưa thăm có f nhỏ nhất; nếu hòa, ưu tiên h nhỏ hơn rồi g lớn hơn. Mỗi cạnh được thư giãn theo:

$$g(v) = g(u) + 1, \quad f(v) = g(v) + h(v)$$

1.5 Hàm findShortestPathMatrix

```
PathNode* findShortestPathMatrix(double adj[100][100],int start,int goal){
    const double INF=1e9;
    double g[100],h[100],f[100]; int parent[100]; bool vis[100];
    for(int i=0;i<100;i++) g[i]=f[i]=INF, h[i]=0, parent[i]=-1, vis[i]=false;

    auto heuristic=[&](int n){ return (double)computeHeuristic(n,goal,adj); };

    g[start]=0; h[start]=heuristic(start); f[start]=g[start]+h[start];

    while(true){
        int u=-1;
        for(int i=0;i<100;i++) if(!vis[i]){
            if(u==-1) u=i;
            else if(f[i]<f[u] || (f[i]==f[u] &&
                (h[i]<h[u] || (h[i]==h[u] && g[i]>g[u])))) u=i;
        }
        if(u==-1 || u==goal){ vis[u]=true; break; }
        vis[u]=true;

        for(int v=0;v<100;v++) if(adj[u][v]>0){
            double newG=g[u]+1, newH=heuristic(v), newF=newG+newH;
            if(newF<f[v]){ g[v]=newG; h[v]=newH; f[v]=newF; parent[v]=u; }
        }
    }

    vector<int> path;
    for(int cur=goal; cur!=-1; cur=parent[cur]) path.push_back(cur);
    reverse(path.begin(), path.end());

    PathNode *head=nullptr,*tail=nullptr;
    for(int v: path){
        PathNode* node=new PathNode(to_string(v),f[v],g[v],h[v]);
        if(!head) head=node; else tail->next=node;
        tail=node;
    }
    return head;
}
```



1.6 Tóm tắt

Hàm triển khai đầy đủ thuật toán A^* với heuristic BFS, tìm được đường đi tối ưu và xuất kết quả dạng danh sách liên kết theo đúng yêu cầu đề bài.

2 Task 2: Shortest Path in 2D Space with Two Heuristics

2.1 Mục tiêu

Task 2 mở rộng thuật toán A* sang không gian 2D. Mỗi đỉnh có tọa độ (x, y) , và heuristic được tính theo hai cách:

- Mode 1: Manhattan distance

$$h = |x_1 - x_2| + |y_1 - y_2|$$

- Mode 2: Euclidean distance

$$h = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Chi phí di chuyển giữa hai đỉnh là giá trị trong ma trận kề `adjMatrix`.

2.2 Thuật toán

A* được áp dụng tương tự Task 1, nhưng:

- $g(v) = g(u) + \text{adjMatrix}[u][v]$ (chi phí thật)
- $h(v)$ lấy theo Manhattan hoặc Euclid tùy mode
- Tên node được in dưới dạng tọa độ (x,y)

2.3 Cài đặt

```
PathNode* findShortestPath2D(double adj[100][100], int coords[100][2],
                             int start, int goal, int mode){
    const double INF = 1e12;
    double g[100], h[100], f[100];
    int parent[100]; bool vis[100];

    for(int i=0;i<100;i++){
        g[i]=f[i]=INF, h[i]=0, parent[i]=-1, vis[i]=false;
    }

    auto heuristic=[&](int n){
        int x1=coords[n][0], y1=coords[n][1];
        int x2=coords[goal][0], y2=coords[goal][1];
        if(mode==1) return (double)(abs(x1-x2)+abs(y1-y2));
        double dx=x1-x2, dy=y1-y2;
        return sqrt(dx*dx + dy*dy);
    };
```

```
g[start]=0; h[start]=heuristic(start); f[start]=g[start]+h[start];

while(true){
    int u=-1;
    for(int i=0;i<100;i++) if(!vis[i]){
        if(u==-1) u=i;
        else if(f[i]<f[u] || (f[i]==f[u] &&
            (h[i]<h[u] || (h[i]==h[u] && g[i]>g[u]))))
            u=i;
    }
    if(u==-1 || u==goal){ vis[u]=true; break; }
    vis[u]=true;

    for(int v=0;v<100;v++) if(adj[u][v]>0){
        double newG=g[u] + adj[u][v];
        double newH=heuristic(v);
        double newF=newG + newH;

        if(newF < f[v]){
            g[v]=newG; h[v]=newH; f[v]=newF; parent[v]=u;
        }
    }
}

vector<int> path;
for(int cur=goal; cur!=-1; cur=parent[cur]) path.push_back(cur);
reverse(path.begin(), path.end());

PathNode *head=nullptr,*tail=nullptr;
for(int v: path){
    string label="(" + to_string(coords[v][0]) + "," + to_string(coords[v][1]) + ")";
    PathNode* node=new PathNode(label,f[v],g[v],h[v]);
    if(!head) head=node; else tail->next=node;
    tail=node;
}
return head;
}
```



2.4 Tóm tắt

Hàm `findShortestPath2D` áp dụng A^* trong không gian 2D với hai heuristic khác nhau, đảm bảo tìm đường đi tối ưu và tạo danh sách liên kết biểu diễn toàn bộ nghiệm.

3 Task 3: Shortest Path in Maze Using A* With 8-Directional Movement

3.1 Mục tiêu

Task 3 yêu cầu áp dụng thuật toán A* để tìm đường đi trong một mê cung kích thước tối đa 100×100 . Một ô hợp lệ mang giá trị 0, ô chướng ngại là 1.

Thuật toán cho phép di chuyển theo 8 hướng:

- 4 hướng cơ bản: Up, Down, Left, Right (cost = 1.0)
- 4 hướng chéo: Up-Left, Up-Right, Down-Left, Down-Right (cost = 1.5)

Kết quả trả về danh sách liên kết `PathNode`, mỗi node lưu:

$$f = g + h$$

g = tổng chi phí di chuyển thực tế

$$h = |x_g - x| + |y_g - y| \quad (\text{Manhattan heuristic})$$

3.2 Cấu trúc dữ liệu

3.2.1 Node kết quả

```
struct PathNode{
    std::string name;
    double f, g, h;
    PathNode* next;
};
```

3.2.2 Node dùng trong A*

```
struct AStarNode{
    int x, y;
    double g, h, f;
    AStarNode* parent;
    std::string direction;
    AStarNode(int x,int y,double g,double h,AStarNode* p,string d)
        : x(x),y(y),g(g),h(h),f(g+h),parent(p),direction(d) {}
};
```

3.3 Heuristic và kiểm tra hợp lệ

Heuristic sử dụng Manhattan distance:

$$h = |x_1 - x_2| + |y_1 - y_2|$$



```
double distance(int x1,int y1,int x2,int y2){  
    return abs(x1-x2) + abs(y1-y2);  
}
```

Kiểm tra một ô có thể đi:

```
bool valid(int x,int y,int m,int n,int maze[100][100]){  
    return x>=0 && x<m && y>=0 && y<n && maze[x][y]==0;  
}
```

3.4 Hướng di chuyển

```
Move dichuyen[8] = {  
    {-1,0,1.0,"Up"}, {1,0,1.0,"Down"},  
    {0,-1,1.0,"Left"}, {0,1,1.0,"Right"},  
    {-1,-1,1.5,"Up-Left"}, {-1,1,1.5,"Up-Right"},  
    {1,-1,1.5,"Down-Left"}, {1,1,1.5,"Down-Right"}  
};
```

3.5 Thuật toán

A* được triển khai theo các bước:

1. Đưa node bắt đầu vào open list.
2. Lặp lại:
 - Chọn node có f nhỏ nhất từ open list.
 - Nếu là node đích $\rightarrow$ dừng.
 - Sinh tối đa 8 node con và tính:

$$g' = g + \text{cost}, \quad h' = \text{Manhattan}, \quad f' = g' + h'$$

- Cập nhật nếu đường đi mới tốt hơn.
3. Truy ngược parent để xây dựng đường đi.

3.6 Hàm findPathInMaze

```
PathNode* findPathInMaze(int maze[100][100],int m,int n,  
                          int startX,int startY,int goalX,int goalY){  
    if(!valid(startX,startY,m,n,maze) || !valid(goalX,goalY,m,n,maze))  
        return nullptr;  
  
    if(startX==goalX && startY==goalY) return nullptr;
```

```
vector<AStarNode*> openList;
bool closed[100][100]={false};

double h_start=distance(startX,startY,goalX,goalY);
openList.push_back(new AStarNode(startX,startY,0,h_start,nullptr,""));

AStarNode* goalNode=nullptr;

while(!openList.empty()){
    int minIdx=0;
    for(int i=1;i<openList.size();i++)
        if(openList[i]->f < openList[minIdx]->f)
            minIdx=i;

    AStarNode* current=openList[minIdx];
    openList.erase(openList.begin()+minIdx);
    closed[current->x][current->y]=true;

    if(current->x==goalX && current->y==goalY){
        goalNode=current; break;
    }

    for(int i=0;i<8;i++){
        int nx=current->x + dichuyen[i].dx;
        int ny=current->y + dichuyen[i].dy;

        if(!valid(nx,ny,m,n,maze) || closed[nx][ny]) continue;

        double newG=current->g + dichuyen[i].cost;
        double newH=distance(nx,ny,goalX,goalY);
        double newF=newG + newH;

        bool inOpen=false;
        for(auto node: openList){
            if(node->x==nx && node->y==ny){
                if(newG < node->g){
                    node->g=newG; node->f=newF;
                    node->parent=current;
                }
            }
        }
    }
}
```

```
        node->direction=dichuyen[i].name;
    }
    inOpen=true; break;
}
}
if(!inOpen)
    openList.push_back(new AStarNode(nx,ny,newG,newH,
        current,dichuyen[i].name));
}
}

if(goalNode==nullptr) return nullptr;

PathNode* head=nullptr;
AStarNode* cur=goalNode;

while(cur->parent!=nullptr){
    PathNode* node=new PathNode();
    node->name=cur->direction; node->g=cur->g;
    node->h=cur->h; node->f=cur->f;
    node->next=head; head=node;
    cur=cur->parent;
}
return head;
}
```

3.7 Tóm tắt

Hàm `findPathInMaze` xây dựng lời giải bằng A* với 8 hướng di chuyển, sử dụng chi phí khác nhau cho bước thường và bước chéo. Output cuối là danh sách liên kết mô tả trình tự các hướng đi tối ưu từ vị trí bắt đầu đến đích.

4 Task 4: Biểu Diễn Mê Cung Dưới Dạng Đồ Thị và Tìm Đường A* Trên Ma Trận Trọng Số

4.1 Mục tiêu

Task 4 mở rộng bài toán tìm đường trong mê cung bằng cách:

- Biểu diễn mê cung dưới dạng đồ thị với tối đa $m \times n$ đỉnh.
- Xây dựng ma trận trọng số cho 8 hướng di chuyển.
- Chạy thuật toán A* trực tiếp trên ma trận kề (*weight matrix*).

Mỗi ô (x, y) được ánh xạ thành một đỉnh duy nhất:

$$\text{vertex} = x \cdot n + y$$

4.2 Cấu trúc dữ liệu

4.2.1 Linked-list kết quả

```
struct PathNode {  
    std::string name;  
    double f, g, h;  
    PathNode* next;  
};
```

4.2.2 Node dùng trong A* cho đồ thị

```
struct AStarNode2 {  
    int vertex, x, y;  
    double g, h, f;  
    AStarNode2* parent;  
    AStarNode2(int v, int x, int y, double g, double h, AStarNode2* p)  
        : vertex(v), x(x), y(y), g(g), h(h), f(g+h), parent(p) {}  
};
```

4.3 Chuyển đổi hệ tọa độ

```
int coordToVertex(int x, int y, int n){ return x*n + y; }  
  
void vertexToCoord(int v, int n, int& x, int& y){  
    x = v / n;  
    y = v % n;  
}
```

4.4 Heuristic

Thuật toán sử dụng Manhattan distance:

$$h = |x_1 - x_2| + |y_1 - y_2|$$

4.5 Xây dựng ma trận trọng số

Mỗi ô sinh tối đa 8 cạnh:

$$\text{cost} = \begin{cases} 1.0 & (4 \text{ hướng thẳng}) \\ 1.5 & (4 \text{ hướng chéo}) \end{cases}$$

```
void buildWeightMatrix(int maze[100][100], int m, int n, double w[100][100]){
    int total = m*n;
    for(int i=0; i<total; i++){
        for(int j=0; j<total; j++){
            w[i][j]=0;

            int dx[]={-1,1,0,0,-1,-1,1,1};
            int dy[]={0,0,-1,1,-1,1,-1,1};
            double cost[]={1,1,1,1,1.5,1.5,1.5,1.5};

            for(int x=0; x<m; x++){
                for(int y=0; y<n; y++){
                    if(maze[x][y]==1) continue;
                    int u = coordToVertex(x,y,n);
                    for(int k=0; k<8; k++){
                        int nx=x+dx[k], ny=y+dy[k];
                        if(nx>=0&&nx<m&&ny>=0&&ny<n && maze[nx][ny]==0){
                            int v = coordToVertex(nx,ny,n);
                            w[u][v] = cost[k];
                        }
                    }
                }
            }
        }
    }
}
```

4.6 Thuật toán tìm đường A* (trên đồ thị)

- Mỗi đỉnh trong đồ thị là một ô hợp lệ trong mê cung.
- A* duyệt qua tất cả các kề của một đỉnh bằng cách đọc hàng tương ứng trong ma trận trọng số.

```
PathNode* findPathInMaze2(int maze[100][100],int m,int n,
    int sx,int sy,int gx,int gy,double w[100][100]){

    buildWeightMatrix(maze,m,n,w);
    int total=m*n;
    int start = coordToVertex(sx,sy,n);
    int goal  = coordToVertex(gx,gy,n);

    vector<AStarNode2*> open;
    bool closed[10000]={false};

    double h0 = manhattanDistance(sx,sy,gx,gy);
    open.push_back(new AStarNode2(start,sx,sy,0,h0,nullptr));

    AStarNode2* goalNode=nullptr;

    while(!open.empty()){
        int best=0;
        for(int i=1;i<open.size();i++)
            if(open[i]->f < open[best]->f)
                best = i;

        AStarNode2* cur=open[best];
        open.erase(open.begin()+best);
        closed[cur->vertex]=true;

        if(cur->vertex==goal){ goalNode=cur; break; }

        for(int v=0; v<total; v++){
            if(w[cur->vertex][v] <= 0) continue;
            if(closed[v]) continue;

            int nx, ny; vertexToCoord(v,n,nx,ny);
            double newG = cur->g + w[cur->vertex][v];
            double newH = manhattanDistance(nx,ny,gx,gy);
            double newF = newG + newH;

            bool inOpen=false;
            for(auto node:open){
```

```
        if(node->vertex==v){
            if(newG < node->g){
                node->g=newG;
                node->h=newH;
                node->f=newF;
                node->parent=cur;
            }
            inOpen=true; break;
        }
    }
    if(!inOpen)
        open.push_back(new AStarNode2(v,nx,ny,newG,newH,cur));
}

if(goalNode==nullptr) return nullptr;

PathNode* head=nullptr;
for(AStarNode2* cur=goalNode; cur!=nullptr; cur=cur->parent){
    string name="("+to_string(cur->x)+","+to_string(cur->y)+")";
    PathNode* node=new PathNode(name,cur->f,cur->g,cur->h);
    node->next=head; head=node;
}
return head;
}
```

4.7 Tóm tắt

Task 4 biểu diễn mê cung dưới dạng đồ thị đầy đủ và cho phép A* hoạt động trực tiếp trên ma trận trọng số. Kết quả đường đi chính xác hơn vì thuật toán sử dụng đầy đủ thông tin đồ thị (bao gồm các bước chéo với chi phí riêng).